

KUBEEDGEINFER

Sharing One Big AI Model Across Small Machines --- and Fixing the Split While It Runs

Most systems choose how to cut the model once. We keep checking, and change the cut whenever the machines change.

Kernel measuring (eBPF)

Kubernetes

Split by layer

Everyday machines

Research Presentation

Author: Institution: Date:

INTRODUCTION

The machines keep changing

1 Why we split the model at all

A big AI model is too large for one laptop or small PC. So we cut it into blocks of layers and give one block to each machine. Where we cut is the most important choice we make.

2 Small machines are not like data centres

Data centre servers are all the same and stay cool. Our machines are laptops and cheap GPU boxes on WiFi. They get hot and slow down, their network speed wanders, and sometimes they switch off.

3 What other systems do today

They measure the machines once, pick a cut, and never look again. That cut is right at the start and gets more and more wrong as the machines change.

Numbers from the test on slide 11 (12 layers, 3 machines).

WHAT THAT COSTS

2.4×

slower on the busiest machine when one machine gets hot and the cut never changes

4.3×

slower when a slow network and a hot GPU happen at the same time

0

answers given after one machine dies, if nothing is re-planned

How other systems split the model

	CONTRIBUTION	WHAT IT DOES NOT DO
EdgeShard IEEE IoT-J, 2024 arXiv:2405.14371	Picks which devices to use and where to cut, with an exact solver. Up to 50% less delay and $2\times$ more work done, tested on 15 real devices.	<i>Measures the devices once, before running. The plan never changes after that.</i>
Galaxy IEEE INFOCOM, 2024 arXiv:2405.17245	Splits the work inside each layer instead of by layer, and overlaps computing with sending. Up to $2.5\times$ less delay.	<i>The plan is made once, before running. Nothing is re-planned while it runs.</i>
Helix ACM ASPLOS, 2025 arXiv:2406.01566	Treats serving on mixed GPUs as a flow problem, and chooses placement and routing together. Up to $3.3\times$ more work done on 24–42 machines.	<i>Built for data centre GPUs. Its solver is far too slow to re-run every few seconds.</i>
TPI-LLM arXiv, 2024 arXiv:2410.00531	Argues that splitting inside layers beats splitting by layer on small devices. Clever memory handling lets 70B models run in small RAM.	<i>Improves memory use, but does not react when a machine slows down.</i>
prima.cpp arXiv, 2025 arXiv:2504.08791	Runs 30–70B models on real home machines. Shares work between CPU and GPU and hides slow disk reads. $5\text{--}17\times$ faster per word than llama.cpp.	<i>Decides the plan when loading, from fixed device specs.</i>

All ten papers are from 2024–2025. Full list on slide 15.

Adapting, sharing out, and eBPF as a control tool

	CONTRIBUTION	WHAT IT DOES NOT DO
LLM Partitioning at the Edge arXiv, 2025 arXiv:2505.02533	Cuts the model into much smaller pieces (single attention heads) and moves them when memory runs low. Within 15–20% of the best possible answer.	<i>Only reacts to memory running low – not to heat or to a slow network.</i>
Parallax arXiv, 2025 arXiv:2509.26182	Serves a model across a pool of volunteer machines of all shapes, with no central cluster in charge.	<i>Spreads the work out well, but measures nothing inside the kernel.</i>
Adaptive Layer Splitting (MBRL) FITEE, Springer, 2025 doi:10.1631/FITEE.2400468	Learns where to cut, and moves the cut as the wireless signal changes. It does keep adapting, which is rare.	<i>The policy must be trained first, and cannot promise the best answer. Only one cut point.</i>
Distributed LLMs & MLLMs Survey arXiv, 2025 arXiv:2503.16585	Reviews the whole field, and lists “adapting to machines that keep changing” as an open problem.	<i>Confirms the gap we are filling. It does not fill it.</i>
Agentic OS / sched_ext arXiv, 2025 arXiv:2509.01245	Loads custom Linux schedulers as eBPF programs while the system runs. Shows eBPF is now used to control things, not only to watch them.	<i>Works on one machine’s CPU tasks. Nothing about splitting a model across machines.</i>

All ten papers are from 2024–2025. Full list on slide 15.

WHAT IS MISSING

What past work is missing

Capability	Data centre serving (Helix)	Splitting on small devices (EdgeShard, Galaxy, prima.cpp)	Learned splitting (MBRL, head-level)	KubeEdgeInfer
Handles machines of different speed	yes	yes	yes	yes
Finds the best cut for the moment	yes, heavy solver	yes	no (learned or rule of thumb)	yes, checked against brute force
Changes the cut while running	only where requests go	no	yes	yes, every 2 seconds
Measures the network in the kernel	no	app-level timers	signal estimate	yes, reads TCP directly
Knows when a machine gets hot	no	no	no	yes, reads GPU and CPU heat
Survives a machine dying	by keeping copies	no	no	yes, re-cuts in 3 seconds
Changes without restarting	not applicable	no	no	yes, moves layers live

No earlier system measures the machines in the kernel and feeds that straight back into the cut, while the model is still answering.

THE PROBLEM

We have N model layers and K machines of different speeds. While the model is answering, their speed, their network delay, and whether they are even alive all keep changing. Keep choosing a cut that makes the slowest machine as fast as possible --- and never restart anything to do it.

1 Measure

Measure network delay and machine heat cheaply, without changing the model code.

2 Decide

Work out the best cut fast enough to redo it often, without flip-flopping when a reading is just noisy.

3 Act

Move layers between running machines without losing requests or restarting anything.

4 Heal

Notice when a machine dies and share its layers among the machines that are left.

THE RULES WE SET OURSELVES

Ordinary Linux only.

No special data centre network, no vendor software, no changes to the model code, and no assuming a machine stays alive.

WHAT IS NEW

What is new in our work

1 We measure the network inside the kernel

Small eBPF programs read the round-trip time Linux already tracks for every connection, and we feed that straight into the cut decision. Other systems guess with app-level timers, or use readings taken before the run.

2 We never stop deciding

Watch, decide, act, heal — every 2 seconds, for as long as the system runs. The cut is something we keep adjusting, not a setting we pick once.

3 We add brakes so it does not flip-flop

The solver gives the exact best cut every time. We only apply a new one if it is at least 15% better and 30 seconds have passed. The same brake handles heat, network and machine failure.

4 We move layers without a restart

Layers move between machines while requests are still in flight. Each plan carries a version number, machines refuse messages from an old plan, and because machines keep no state we can simply replay the request.

5 We show the gain comes from live measuring

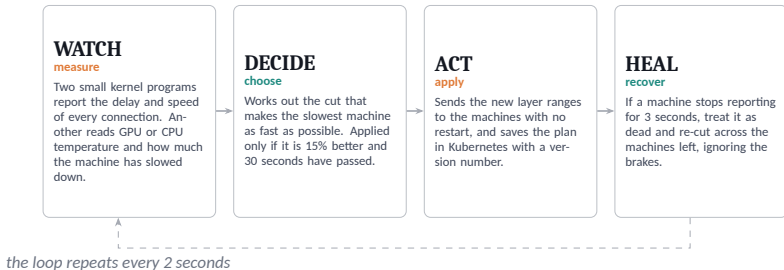
We built a “measure once” mode that uses the same solver and the same code, but freezes what it measured. So the improvement we report comes from measuring all the time, not from a better solver.

THE MAIN NEW IDEA

We did not invent a new splitting algorithm. We took the standard one and put it inside a loop that keeps measuring the real machines and never stops.

Items 1–3 are the main new ideas.

How the loop works



Staying correct: every plan carries a version number. Machines refuse messages from an old plan. The router then fetches the new plan and replays the request from the start. Machines keep no state, so replaying is always safe.

The maths, and how we keep it steady

The goal

make the slowest machine as fast as possible

$$\text{machine time} = \frac{\text{layers} \times \text{time per layer}}{\text{speed}} + \text{network delay}$$

Each machine gets a block of layers next to each other, and together they cover all N layers. A machine given zero layers costs nothing at all — so the solver can skip a machine whose network has become too slow to be worth using.

How we solve it

$\text{best}[j][w]$ = the best we can do for the first j layers on w machines

This is a classic problem with a known exact answer. For 12 layers and 3 machines it takes about 1,700 steps, so redoing it every 2 seconds costs nothing. We checked it against brute force on 200 random cases.

Why a solver alone is not enough

Measurements jump around. If we applied the best answer every 2 seconds, the system would keep moving layers for tiny gains. Three rules fix that:

- **Must be clearly better**

Only change if the new cut beats the current one, measured right now, by 15%.

- **Wait between changes**

At most one change every 30 seconds, so a short spike cannot start a wobble.

- **Except when a machine dies**

If the set of machines changes, ignore both rules and re-cut at once. Being correct matters more than being steady.

Both numbers can be changed while the system is running.

What we built

- **Agent on each machine** Go and C

Runs on every machine with kernel access. Loads the eBPF programs, watches only the ports we care about, and reports network delay plus GPU state. GPU readings can be simulated, taken from CPU heat sensors, or read from a real NVIDIA card.

- **Central controller** Go

Collects all the readings, runs the decider, saves the plan in Kubernetes and sends it to the machines. A watchdog forces a re-cut if a machine goes quiet for 3 seconds.

- **Workers and router** Python

Each worker holds a block of layers and keeps no state. Two backends: a fake one for testing, and real GPT-2. Runs on CPU or GPU. The router sends each request through the machines in order.

- **Test harness** Python

Five fault situations × three modes. Measures idle time, words per second, and time to the first word. Also tries different brake settings.

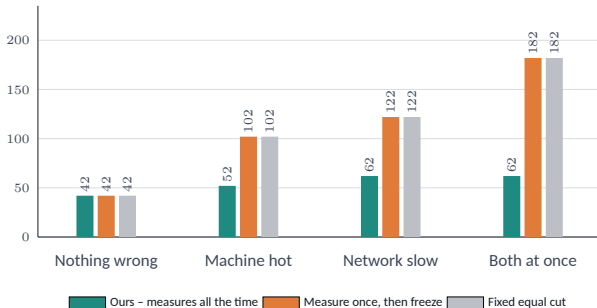
What is real, what is not

- The eBPF part is real, not simulated. It is built against the running kernel and loaded into it.
- On a single test host you get real eBPF, but a fake network and a fake GPU.
- Our scripts turn a few laptops and GPU boxes on the same network into one cluster, with no config editing.
- Real NVIDIA readings on GPU machines; real CPU heat readings on laptops with no GPU.
- The split-up GPT-2 gives exactly the same text as running it on one machine.

RESULTS

How slow the busiest machine gets

Time on the slowest machine, per word (ms) – lower is better



49% better than both when a machine gets hot

49% better than both when the network slows

66% better than both when both happen at once

When a machine dies, the fixed cut has no plan for the machines left, so everything stops. Ours re-cuts and keeps going at 62 ms.

Measured with *bench/dpsim*: 12 layers, 3 machines, 10 ms per layer, one machine slowed to 0.4x, 80 ms network delay.

RESULTS

What the solver actually decides

	layers given to machine 1 / 2 / 3			
All machines healthy	0-4	4-8	8-12	42 ms
3 machines, all fine				
Machine 2 gets hot	0-5	5-7	7-12	52 ms
speed drops to 0.4x				
Network to machine 2 slows	0-6	empty	6-12	62 ms
delay 2 ms -> 82 ms				
Machine 3 dies	0-6	6-12		62 ms
3 machines -> 2				

Words per second, time to the first word, and real idle time come from bench/run.py, which needs a running cluster.

It can skip a machine

When the network to machine 2 slows to 80 ms, the solver gives it zero layers. A machine with no layers costs nothing at all, so the router skips it and the other two take all 12 layers. Nobody told it to do this — it falls out of the maths.

- **It finds the best answer**

The solver matched brute force on 200 random test cases.

- **It gives the same answers**

GPT-2 split over 3 machines produces exactly the same text as one machine.

- **How good the estimate is**

Our predicted time is always about $3\times$ lower than the real time. Good for ranking cuts, not for predicting real speed.

COMPARISON

How we compare with the closest systems

	EdgeShard (IoT-J 24)	Galaxy (INFOCOM 24)	Helix (ASPLOS 25)	prima.cpp (2025)	KubeEdgeInfer
How the model is split	by layer	inside layers	by layer + routing	ring of layers	by layer
How the cut is chosen	exact solver	rule of thumb	heavy solver	custom scheduler	exact solver
Where its numbers come from	measured before	measured before	measured before	device specs	kernel + GPU, live
Re-cuts while running	no	no	only routing	no	every 2 seconds
Reacts to machines getting hot	no	no	no	no	yes
Survives a machine dying	no	no	by keeping copies	no	yes, within 3 seconds
What it runs on	15 small devices	small devices	24–42 GPU servers	home machines	laptops + GPU boxes
Improvement they report	50% less delay	2.5× less delay	3.3× more work	5–17× faster words	49–66% better*

* We measure how slow the busiest machine gets, against a fixed cut and a measure-once version of our own system. That is not the same measurement as the other columns.

Where to cut the model is a decision you should keep making, not one you make once.

We keep a standard, exact solver running all the time. It is fed by network delays read from the kernel and by real temperature readings, and it moves layers on a live system with no restart. On a set of ordinary machines, that is the difference between a plan that was right once and a plan that stays right.

1 The exact answer is cheap

Working out the best cut takes microseconds. There is no reason to do it only once.

2 The brakes are what make it usable

One “must be clearly better” rule and one “wait a bit” rule handle heat, network and machine failure all the same way.

3 Kernel measuring is free

eBPF sees what really went over the network, not what the program thinks it sent.

Next steps

- Support a Llama-size model, so two 8 GB cards can hold a real modern model between them.
- Run heavy load for a long time, to see a GPU really slow down instead of flat lines.
- Allow cuts that are not one solid block per machine, as in [arXiv:2505.02533](https://arxiv.org/abs/2505.02533).
- Calibrate the cost estimate so it predicts real speed, not only the right ranking.

REFERENCES

All works cited

1. EdgeShard: Efficient LLM Inference via Collaborative Edge Computing. M. Zhang, J. Cao, X. Shen, Z. Cui. *IEEE Internet of Things Journal*, 2024. arXiv:2405.14371.
2. Galaxy: A Resource-Efficient Collaborative Edge AI System for In-situ Transformer Inference. *IEEE INFOCOM*, 2024. arXiv:2405.17245.
3. Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow. *ACM ASPLOS*, 2025. arXiv:2406.01566.
4. TPI-LLM: Serving 70B-scale LLMs Efficiently on Low-resource Edge Devices. 2024. arXiv:2410.00531.
5. Prima.cpp: Fast 30–70B LLM Inference on Heterogeneous and Low-Resource Home Clusters. 2025. arXiv:2504.08791.
6. Large Language Model Partitioning for Low-Latency Inference at the Edge. 2025. arXiv:2505.02533.
7. Parallax: Efficient LLM Inference Service over Decentralized Environment. 2025. arXiv:2509.26182.
8. Adaptive layer splitting for wireless large language model inference in edge computing: a model-based reinforcement learning approach. *Frontiers of Information Technology & Electronic Engineering*, Springer, 2025. doi:10.1631/FITEE.2400468.
9. Distributed LLMs and Multimodal Large Language Models: A Survey on Advances, Challenges, and Future Directions. 2025. arXiv:2503.16585.
10. Towards Agentic OS: An LLM Agent Framework for Linux Schedulers. 2025. arXiv:2509.01245.

Tools used: cilium/ebpf · Kubernetes controller-runtime · NVIDIA NVML · k3s · Hugging Face Transformers (GPT-2).